

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 6 — Projects e Context Engineering

Una struttura che un agente sa navigare

Claude naviga come un ingegnere

Ciò che non sta nel `CLAUDE.md`, Claude lo cerca da sé: come, e che cosa lo aiuta

- Nel `CLAUDE.md` va solo ciò che Claude **non trova** leggendo il repository: il resto lo trova da sé, navigando il progetto
- Un articolo di Anthropic: «Claude Code navigates a codebase the way a software engineer would: it traverses the file system, reads files, uses grep». Con Grep cerca i file e si costruisce il contesto **da solo**
- E rende meglio «when Claude has enough starting context to know **where to look**»
- Cartelle, nomi e date dei file sono **segnali**, per le persone e per gli agenti: un `test_utils.py` in `tests/` non ha lo scopo di un file con lo stesso nome in `src/core_logic/`
- Nel gestionale del corso le cartelle parlano già: `src/` la parte nuova, `legacy/` il vecchio gestionale, `tests/Unit/` i test senza database, `tests/Integrazione/` quelli con SQLite in memoria

Dove lo avviate decide che cosa carica

La cartella di partenza; e un `CLAUDE.md` per cartella, o una regola con `paths`

All'avvio Claude carica i `CLAUDE.md` della cartella da cui parte e di **quelle sopra**: dalla radice, quello della radice; da una sottocartella, quello e quelli di tutte le cartelle superiori, così il contesto della radice non si perde. Gli altri, **quando servono**

avviato da	vede	carica all'avvio	va bene per
la radice del repository	ogni file	il `CLAUDE.md` della radice; quelli delle sottocartelle quando servono	compiti che toccano più parti
una sottocartella	solo quel sottoalbero, finché non gli date di più	quello della cartella e di tutte le superiori	il lavoro su una parte sola

per una parte	dove sta	quando si carica	va bene se
un `CLAUDE.md` per cartella	nella cartella, accanto al codice	all'avvio, se partite da lì; altrimenti quando serve	ogni cartella ha chi ne cura le convenzioni
una regola con `paths`	in `.claude/rules/`, alla radice	quando Claude lavora su un file che corrisponde ai suoi `paths`	volete tutto in un posto; percorsi sparsi; un aspetto trasversale

Dalla documentazione, «Set up Claude Code in a monorepo or large codebase»; l'aspetto trasversale da «Steering Claude Code»

Il file radice e i documenti di riferimento

Che cosa sta nel file radice, e come ci entra un documento

- Il **file radice**, per l'articolo sulle codebase grandi: «pointers and critical gotchas only; everything else drifts into noise». Il `CLAUDE.md` come panoramica, o come «an index pointing to other files». Il team di Claude Code lo tiene corto, perché ogni code review lo carica, e da lì rimanda ad altri documenti
- Una mappa delle cartelle serve solo dove il codice **non sta in una struttura convenzionale**: un file Markdown leggero alla radice, una riga per cartella
- Un documento di riferimento, **tre modi**:
  - **importato** con `@percorso`: si carica sempre, con il `CLAUDE.md` che lo importa; organizza, ma **non riduce** il costo
  - **nominato** a parole: Claude lo vede solo se decide di aprirlo; la documentazione lo dice per `AGENTS.md`
  - in una **skill**, se è una procedura in più passi; in una **regola** con `paths`, se conta per una parte del codice
- Gli ADR (Architecture Decision Records) **non compaiono** nelle pagine di Anthropic lette per il corso; la documentazione mette le **decisioni architetturali** fra i contenuti del `CLAUDE.md`. Una cartella di decisioni, nominata nel `CLAUDE.md`, è un esempio del corso
- A voce: le convenzioni del gestionale, le importereste, le nominereste o ne fareste una regola?

Il file radice e i documenti di riferimento

Che cosa sta nel file radice, e come ci entra un documento

- Nel laboratorio del modulo (M6-E02 sul gestionale, M6-E03 sulla rete), il `CLAUDE.md` di riferimento del gestionale ha le convenzioni **nominate**, non importate: una riga basta. La sezione 9, del codice legacy, sta in una **regola** con `paths`

righe scelte, a capo per la slide; «[…]» segna un taglio

```
# Gestionale CiaspoLink

Il gestionale PHP sta in `progetti/gestionale-php/`;
`tools/php.sh` esegue PHP, Composer e PHPUnit anche dove PHP
non c'è.
[…]
## Convenzioni che il codice non dice
[…]
- Il soggetto segue `docs/convenzioni.md`, 11.1:
  `tipo(ambito): descrizione`, all'indicativo presente.

## Insidie

- `database/dati/` e `composer.lock` si rigenerano, non si
  modificano: i dati con `python3 genera.py`, le dipendenze con
  Composer.
```

Il codice che non va letto

Il rumore che Claude non deve aprire

- Le ricerche di Claude rispettano `.gitignore`: ciò che vi è elencato, come `node_modules/`, `dist/` e `build/`, resta **fuori dai risultati**. Nel gestionale è il caso di `vendor/`, le dipendenze di Composer
- Il codice generato, o una libreria di terzi, se sta **nel repository** si esclude con una regola `Read` in `permissions.deny`, che impedisce a Claude di aprirlo
- Nel `CLAUDE.md` d'esempio della documentazione, un'insidia con il suo rimedio: «Never edit files under packages/*/generated/. Run npm run codegen in the package instead.»
- Nel gestionale `database/dati/`, nella rete `configurazioni/`, `log/`, `inventario.csv` e `allarmi.csv`: li scrive `genera.py`, e leggerli serve. I `CLAUDE.md` di riferimento del laboratorio li mettono fra le **insidie**, si rigenerano e non si modificano a mano, e regole `deny` su `Edit` ne bloccano la modifica con gli strumenti di file

Una struttura che parla da sola, e un file radice che dice dove guardare.